



18. Python: Lambda Functions

More about Python and its strongest ability → Working with functions

Previous Section:

 [17. Python: Random Numbers](#)

Contents:

[1. Lambda in Python](#)

[2. Common Lambda Functions](#)

[3. Some Properties](#)

[Summary](#)

[References](#)

Sometimes, writing a full function feels a bit excessive. You create a function. Write `return`. Only to use it once.

And that is exactly why Python has something called a **lambda function**. A lambda function is a small anonymous function designed for short operations.

Instead of writing:

```
def square(x):  
    return x * x
```

You can simply write:

```
square = lambda x: x * x
```

Cleaner. Shorter. Faster to write.

But there is also an important limitation: Lambda functions can only contain **one expression**. Meaning no multiple statements, no loops written traditionally, no large blocks of logic. They are designed for quick operations, not large programs.

And unlike normal functions, the result is returned automatically. No `return` keyword is needed.

1. Lambda in Python

The general syntax looks like this:

```
lambda parameters: expression
```

Example:

```
add = lambda a, b: a + b
print(add(3, 5))
```

8

The expression after `:` is automatically returned.

- Multiple Parameters: Lambda functions can take multiple inputs.

```
multiply = lambda x, y, z: x * y * z
print(multiply(2, 3, 4))
```

24

- Lambda with Strings

```
greet = lambda name: "Welcome " + name
print(greet("Iris"))
```

Welcome Iris

- Lambda with Mathematical Expressions

```
equation = lambda x: x ** 2 + 5
print(equation(3))
```

14

2. Common Lambda Functions

- Using If-Else in Lambda: Lambda functions support inline conditional expressions.

```
check = lambda age: "Adult" if age >= 18 else "Minor"

print(check(20))
print(check(12))
```

Adult

Minor

This is useful for quick variable assignments.

- Using Iteration with Lambda: Lambda is commonly used together with functions like: `map()` , `filter()` , `reduce()`

Example with `map()`

```
numbers = [1, 2, 3, 4]

squared = list(map(lambda x: x ** 2, numbers))
print(squared)
```

[1, 4, 9, 16]

Example with `filter()`

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even = list(filter(lambda x: x % 2 == 0, numbers))  
print(even)
```

[2, 4, 6]

Example with `reduce()`

```
from functools import reduce
```

```
# Summing a list
```

```
number1 = [1, 2, 3, 4]
```

```
total = reduce(lambda x, y: x + y, number1)
```

```
print("Total:", total)
```

```
# Finding the maximum value
```

```
number2 = [5, 12, 8, 20, 3]
```

```
max_val = reduce(lambda a, b: a if a > b else b, number2)
```

```
print("Max Value:", total)
```

Total: 10

Max Value: 10

- Define a Function then Use Lambda: Sometimes, lambda works nicely with normal functions.

```
def power(n):
    return lambda x: x ** n

square = power(2)
cube = power(3)

print(square(4))
print(cube(2))
```

16
8

This creates functions dynamically.

- Sorting a List with Lambda: Lambda is heavily used in sorting.

```
students = [("John", 75), ("Emma", 92), ("Alex", 81)]

students.sort(key=lambda x: x[0])
print(students)
```

[('Alex', 81), ('Emma', 92), ('John', 75)]

The lambda tells Python: ***"Sort using the first value."***

- Sorting Tuples Based on Second Number. Suppose the first numbers are identical.

```
data = [(1, 5), (1, 2), (1, 9), (1, 1)]

result = sorted(data, key=lambda x: x[1])
print(result)
```

[(1, 1), (1, 2), (1, 5), (1, 9)]

- Reversing with Lambda

```
words = ["apple", "banana", "orange"]  
  
words.sort(key=lambda x: x[::-1])  
print(words)
```

Possible Output: ['banana', 'orange', 'apple']

The slice: `[::-1]` reverses the string. And since the `.sort()` method sorts the list by evaluating the **last letter** of each word.

- "apple" becomes "elppa" (starts with **e**)
- "banana" becomes "ananab" (starts with **a**)
- "orange" becomes "egnaro" (starts with **e**)

⇒ Since "**a**" comes before "**e**", "banana" goes first.

- Returning Multiple Results: Lambda can return tuples.

```
values = lambda x: (x, x ** 2, x ** 3)  
print(values(3))
```

(3, 9, 27)

3. Some Properties

- In functions, once `return` executes, the function immediately stops.

```
def example():
    print("Start")
    return

    print("End")

example()
```

Start

The line after `return` never runs. So in many ways, `return` behaves similarly to a break from the function itself.

- De Morgan's Rule: This rule becomes extremely important in complicated conditional problems. The idea is:

$$\neg(A \wedge B) = \neg A \vee \neg B \quad \text{And:} \quad \neg(A \vee B) = \neg A \wedge \neg B$$

Example:

```
x = 5

# Equivalent logic:
# if x <= 0 or x >= 10:
#     print("Outside range")

if not (x > 0 and x < 10):
    print("Outside range")
else:
    print("Inside range")
```

Inside range

De Morgan's Rule helps simplify conditions significantly.

- Nested in Nested Loops: Lambda functions can appear inside loops and even nested structures.

```
matrix = [[1, 2], [3, 4]]
double = lambda x: x * 2

for row in matrix:
    result = list(map(double, row))
    print(result)
```

[2, 4]

[6, 8]

This is common in: Data processing; AI pipelines; Matrix operations; Functional programming

Summary

This section introduced Python's lambda functions — small anonymous functions designed for quick operations. We learned that lambda functions:

- Contain only one expression
- Automatically return results
- Work well with sorting and iteration
- Are commonly used with `map()` and `filter()`
- Can simplify conditional logic dramatically

We also explored important programming concepts connected to functions:

- `return` immediately stops execution
- De Morgan's Rule simplifies complex conditions
- Lambda functions can integrate into nested loops and data pipelines

Lambda functions may look small, but they are heavily used in: Data Science; Machine Learning; Functional Programming; Automation; AI systems

Because sometimes, the cleanest solution is not a giant function. Sometimes, a single expression is enough.

References

<https://www.geeksforgeeks.org/python/python-lambda-anonymous-functions-filter-map-reduce/>

<https://www.datacamp.com/tutorial/python-lambda-functions>

<https://www.codecademy.com/article/python-lambda-function>

Next Section:

[</> 19. Python: Regular Expressions](#)